

GMAO AI – Maintenance Industrielle Intelligente

Rapport Technique de Stage

Analyse de donnees et modeles predictifs pour la GMAO

Fevrier 2026

Table des matières

1	Introduction	2
2	Donnees	2
2.1	Sources	2
2.2	Qualite des donnees	2
2.3	Nettoyage et integration	2
3	Feature Engineering	3
3.1	Categories de features	3
3.2	Variables cibles	3
4	Modeles	3
4.1	Detection d'anomalies (non supervise)	4
4.2	Prediction de panne a 30 jours (supervise)	5
4.3	Prediction de besoin en pieces de rechange (supervise)	6
5	Analyses complementaires	7
5.1	Efficacite de la maintenance preventive	7
5.2	Gestion des pieces de rechange	7
6	Architecture et deploiement	8
6.1	Architecture logicielle	8
6.2	Application web	8
6.3	Deploiement	9
7	Recommandations	9
8	Limites et perspectives	10
8.1	Limites	10
8.2	Perspectives	10
9	Conclusion	10

1 Introduction

Ce projet de stage porte sur l'analyse des données issues d'un système de GMAO (Gestion de Maintenance Assistée par Ordinateur) couvrant 5 sites industriels. L'objectif est de construire un système d'aide à la décision basé sur le Machine Learning, capable de :

- Détecter des anomalies dans les ordres de maintenance
- Prédire les pannes équipement à 30 jours
- Anticiper les besoins en pièces de rechange
- Évaluer l'efficacité de la maintenance préventive
- Fournir des recommandations opérationnelles chiffrées

Le livrable final est une application web Streamlit déployée, exposant les résultats d'analyse, les visualisations et une interface de prédiction interactive.

2 Données

2.1 Sources

Trois fichiers Excel constituent les données brutes :

Fichier	Contenu	Lignes	Colonnes
Ordres de travail.xlsx	Maintenance corrective	2 035	51
OT_Preventifs.xlsx	Maintenance préventive	9 149	40
a-w.xlsx	Demandes de pièces de rechange	12 573	22

TABLE 1 – Datasets sources

Période couverte : 1^{er} janvier – 31 mars 2025 (89 jours).

Sites : TM1, PPR, TM2, MEDHUB, TMSA.

Actifs uniques : 7 090 équipements suivis.

2.2 Qualité des données

- **Correctif :** 0 doublons. Colonnes avec valeurs manquantes significatives : `cout_bc` (100%), `commentaire_ouverture` (98%), `cout_pdr` (78%).
- **Preventif :** 0 doublons. Colonnes de rejet/commentaires majoritairement vides (>97%).
- **Pièces de rechange :** 437 doublons supprimés. `description` manquante à 97%.

2.3 Nettoyage et intégration

Le pipeline de preprocessing (`src/preprocessing.py`) effectuée :

1. Correction de l'encodage texte (caractères accentués)
2. Standardisation des formats de date
3. Conversion des booléens (Oui/Non → True/False)
4. Remplissage des coûts manquants par 0

5. Calcul de metriques derivees : `time_to_closure_hours`, `time_to_start_hours`
6. Integration correctif-pieces de rechange (jointure sur `id_ordre_travail`)
7. Construction d'un referentiel actifs (`asset_master`) et d'une timeline unifiee

Note importante : Les colonnes temporelles different entre les jeux de donnees. Le correctif utilise `date_creation_ot`, tandis que le preventif utilise `date_debut` et `date_debut_planifiee`. Le correctif utilise `cout_total`, le preventif `cout`. Cette difference a cause des erreurs (`KeyError`) dans les premieres versions du code et a necessite une attention particuliere lors de l'integration.

3 Feature Engineering

A partir des donnees nettoyees, 95 features et 5 variables cibles ont ete construites (`src/feature_engineering.py`). Le dataset final compte 2035 echantillons.

3.1 Categories de features

Categorie	Nb	Exemples
Temporelles	8	<code>creation_hour</code> , <code>days_since_last_maintenance</code>
Equipement	8	<code>equipment_failure_count</code> , <code>equipment_age_days</code>
Localisation	3	<code>site_failure_count</code> , <code>zone_failure_count</code>
Pieces de rechange	3	<code>has_spare_parts</code> , <code>spare_parts_cost_ratio</code>
Anomalies	21	20 indicatrices binaires + <code>anomaly_recurrence_count</code>
Urgence	4	<code>urgence_Fort</code> , <code>famille_high_urgency_rate</code>
Prestataire	3	<code>provider_avg_cost</code> , <code>provider_work_volume</code>
Fenetres glissantes	4	<code>rolling_7d_failures</code> , <code>rolling_30d_avg_cost</code>
Categoriques encodees	7	<code>site_encoded</code> , <code>famille_encoded</code> (label encoding)

TABLE 2 – Repartition des 95 features par categorie

3.2 Variables cibles

Variable	Type	Taux positif
<code>will_fail_within_30_days</code>	Classification binaire	18.9%
<code>will_need_spare_parts</code>	Classification binaire	26.2%
<code>anomaly_type_encoded</code>	Classification multi-classe	268 types
<code>spare_parts_cost_target</code>	Regression	–
<code>intervention_duration_target</code>	Regression	–

TABLE 3 – Variables cibles construites

4 Modeles

Trois modeles ont ete developpes, tous implements dans `src/models/` avec une API uniforme (`fit`, `predict`, `save`, `load`). Le premier (section 4.1) est un modele **non su-**

pervise : il ne dispose d’aucune etiquette “anomalie” dans les donnees et apprend a detecter les comportements inhabituels a partir de la distribution des features. Les deux suivants (sections 4.2 et 4.3) sont des modeles **supervises** : ils s’entraignent sur des variables cibles binaires construites a partir de l’historique (`will_fail_within_30_days`, `will_need_spare_parts`) et apprennent a reproduire cette relation sur des donnees in-edites.

4.1 Detection d’anomalies (non supervise)

Approche : Ensemble de deux methodes non supervisees avec vote par consensus.

- **Isolation Forest** : isole les points aberrants en partitionnant aleatoirement l’espace des features. Les anomalies sont isolees plus rapidement (profondeur d’arbre faible).
- **One-Class SVM** : apprend la frontiere de la distribution “normale” dans un espace de haute dimension (noyau RBF).

Justification du choix : Ces deux methodes sont complementaires et particuliere-ment adaptees a la detection d’anomalies en maintenance. L’Isolation Forest excelle sur les anomalies globales (points isoles dans l’espace des features) avec une complexite lineaire $O(n)$, ce qui le rend efficace sur de grands volumes. Le One-Class SVM capture mieux les anomalies locales proches de la frontiere de decision grace a son noyau RBF, qui projette les donnees dans un espace de dimension superieure. Leur combinaison par consensus reduit les faux positifs : le taux d’accord de 93.7% confirme la coherence des detections.

Consensus : Un echantillon est marque comme anomalie si *au moins une* des deux methodes le detecte. L’anomalie est dite “haute confiance” si les deux methodes concordent.

16 features numeriques sont utilisees : couts, durees, metriques equipement, fenetres glissantes.

Metrique	Valeur
Echantillons totaux	2 035
Anomalies (consensus)	268 (13.2%)
Anomalies haute confiance	140 (6.9%)
Accord entre methodes	93.7%

TABLE 4 – Resultats de la detection d’anomalies

Les anomalies detectees presentent un cout moyen 14x superieur (45 454 vs 3 272 MAD), une duree d’intervention 7.6x superieure et un temps de cloture 7.1x superieur aux ordres normaux.

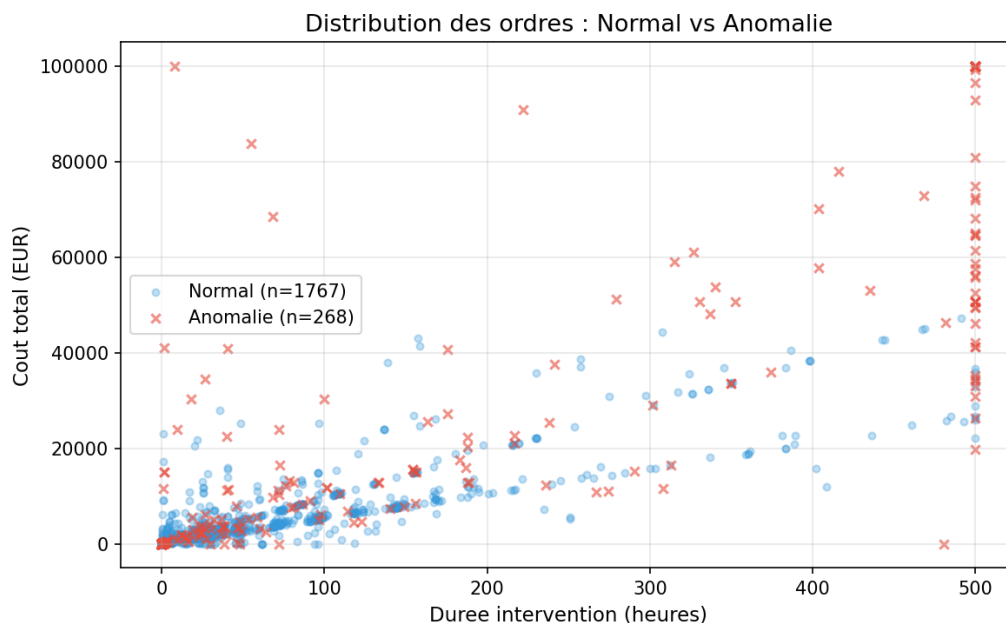


FIGURE 1 – Distribution des ordres de travail selon le cout et la duree d'intervention. Les anomalies (croix rouges) se concentrent dans les zones de cout et/ou duree elevees.

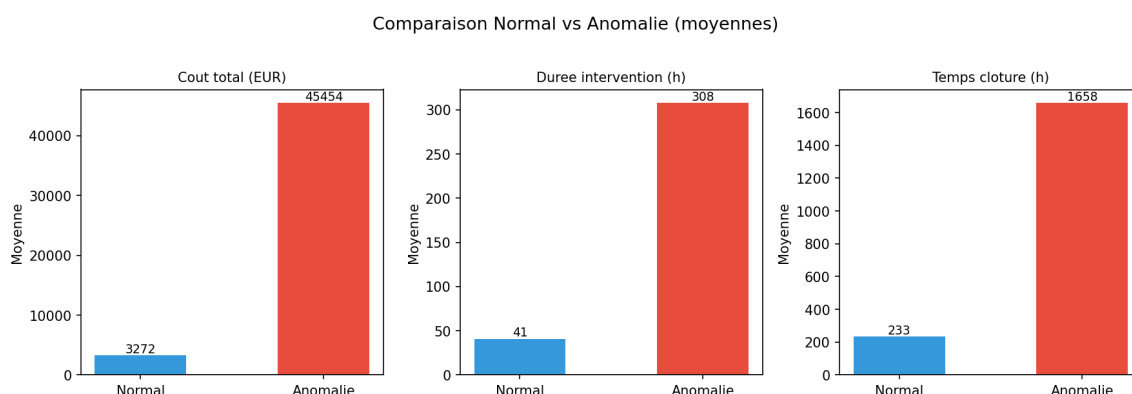


FIGURE 2 – Comparaison des moyennes entre ordres normaux et anomalies. Les anomalies presentent des valeurs significativement superieures sur les trois metriques clés.

Validation : Etant non supervisee, ce modele ne dispose pas de metriques classiques (precision, recall). Sa fiabilite repose sur trois elements : (1) le taux d'accord de 93.7% entre deux methodes mathematiquement independantes, (2) la separation statistique nette entre les groupes normal/anomalie (différences de 7x a 14x sur les metriques clés), et (3) la coherence avec le sens metier — un ordre coutant 45 000 MAD avec 300+ heures d'intervention est objectivement anormal par rapport a une mediane de 230 MAD et 2.5 heures. L'absence de verite terrain reste une limite inherente a l'approche non supervisee.

4.2 Prediction de panne a 30 jours (supervise)

Algorithme : Random Forest Classifier (200 arbres, profondeur max 12, `class_weight='balanced'`)

Justification : Le Random Forest a ete choisi pour sa robustesse face aux desequilibres de classes (18.9% de positifs), son interpretabilite via les importances de features,

et sa capacite a capturer des interactions non lineaires sans risque important de suraprentissage.

Preprocessing : Standardisation (StandardScaler), split 80/20 stratifie.

Metrique	Test	CV (5-fold)
Precision	0.756	–
Recall	0.844	–
F1-score	0.797	–
ROC AUC	0.973	0.971 ± 0.005
Accuracy	0.919	–

TABLE 5 – Performance du modele de prediction de panne

Matrice de confusion (test) :

	Predit : Non	Predit : Oui
Reel : Non	309	21
Reel : Oui	12	65

Top 5 features :

1. `equipment_failure_count` (37.0%) – nombre de pannes historiques
2. `anomaly_recurrence_count` (17.0%) – recurrence des anomalies
3. `equipment_age_days` (2.9%) – age de l'equipement
4. `rolling_30d_failures` (2.5%) – pannes dans les 30 derniers jours
5. `famille_avg_cost` (2.3%) – cout moyen de la famille

La predominance de `equipment_failure_count` confirme l'intuition que les equipments ayant deja connu des pannes sont les plus a risque de recidive.

4.3 Prediction de besoin en pieces de rechange (supervise)

Algorithme : Gradient Boosting Classifier (150 arbres, profondeur max 8, learning rate 0.1).

Justification : Le Gradient Boosting construit ses arbres de maniere sequentielle, chaque arbre corrigeant les erreurs du precedent. Cette approche iterative le rend particulierement performant sur les problemes de classification tabulaire avec des features de natures differentes (couts, durees, indicatrices). Il surpasse generalement le Random Forest lorsque les donnees presentent des relations complexes entre features, au prix d'un temps d'entrainement legerement superieur.

Contrainte de deploiement : Le `GradientBoostingClassifier` de scikit-learn utilise en interne des fonctions de perte compilees en Cython (`CyHalfBinomialLoss`). Ces binaires sont specifiques a l'OS et a l'architecture : un modele serialise (pickle) sous Windows ne peut pas etre charge sous Linux, meme avec la meme version de scikit-learn. Pour contourner ce probleme, le modele est **re-entraine a la volee** au demarrage de l'application Streamlit, a partir du dataset de features (2035 lignes). L'entrainement prend environ 2 secondes et est mis en cache (`@st.cache_resource`) pour ne s'executer qu'une seule fois par session.

Metrique	Test	CV (5-fold)
Precision	0.961	–
Recall	0.925	–
F1-score	0.943	–
ROC AUC	0.990	0.989 ± 0.006
Accuracy	0.971	–

TABLE 6 – Performance du modele de prediction de pieces de rechange

Top 5 features :

1. `cout_pdr` (79.3%) – cout historique des pieces de rechange
2. `famille_spare_parts_usage_rate` (5.7%) – taux d'utilisation pieces par famille
3. `time_to_closure_hours` (2.6%) – temps de cloture
4. `duree_intervention_par_heure` (2.5%) – duree de l'intervention
5. `groupe_encoded` (1.9%) – groupe d'equipement

La forte dominance de `cout_pdr` (79.3%) est caracteristique du Gradient Boosting : les premiers arbres capturent l'essentiel du signal, les suivants affinent les residus. Cela indique que le cout historique des pieces est de loin le meilleur predicteur du besoin futur.

5 Analyses complementaires

5.1 Efficacite de la maintenance preventive

Indicateur	Valeur
Taux de conformite (dans les delais)	26.5%
Ecart moyen par rapport au planning	23.7 jours
Ecart median	7.6 jours
Intervalle optimal calcule	43 jours
ROI de la maintenance preventive	−90.4%

TABLE 7 – Indicateurs d'efficacite de la maintenance preventive

Le ROI negatif s'explique par le cout tres eleve de la maintenance preventive (410M MAD sur 89 jours) par rapport aux couts correctifs evites (40M MAD estimes). Cela suggere soit un surinvestissement preventif, soit une mauvaise allocation des ressources. Seules 26.5% des interventions preventives sont realisees dans les delais prevus.

5.2 Gestion des pieces de rechange

- **Analyse Pareto** : 144 articles (11% du total) representent 80% des couts.
- **Taux de rupture de stock** : 49% des demandes concernent des articles en rupture.
- **Taux de rejet** : 16.4% des demandes sont rejetees.
- **Articles a reapprovisionner** : 911 articles identifies.

— **Cout previsionnel 90 jours** : 1 256 421 MAD.

Les familles les plus touchees par les ruptures sont la plomberie (1 755), l'electricite (1 546) et la menuiserie (1 349).

6 Architecture et deployment

6.1 Architecture logicielle

Le projet suit une architecture modulaire :

```
gmao-ai-project/
src/
  preprocessing.py           # Chargement, nettoyage, integration
  feature_engineering.py     # Construction des 95 features
  models/
    anomaly_detector.py      # Isolation Forest + One-Class SVM
    failure_predictor.py     # RandomForest (panne + pieces)
    effectiveness.py         # ROI, conformite, intervalles
    spare_parts_forecaster.py # Pareto, previsions,
                             reapprovisionnement
  app/
    streamlit_app.py         # Dashboard interactif (7 pages)
    run_pipeline.py          # Pipeline en 4 etapes
```

Chaque module de modele expose une API uniforme : `fit()`, `predict()`, `save()`, `load()`, permettant la reutilisation et le re-entrainement independant.

Le pipeline complet s'execute via :

```
python run_pipeline.py           # Toutes les etapes
python run_pipeline.py --step 3 # Une etape specifique
```

6.2 Application web

Le dashboard Streamlit comprend 7 sections :

1. **Accueil** – Presentation du projet et guide de navigation
2. **Vue d'ensemble** – KPIs et resume des performances
3. **Exploration des donnees** – Qualite, tendances, MTBF, Pareto, anomalies
4. **Feature Engineering** – Visualisation des 95 features et correlations
5. **Modeles & Analyses** – Metriques, matrices de confusion, importances
6. **Prediction Interactive** – Formulaire de saisie avec predictions en temps reel
7. **Recommandations** – Constats et ROI des initiatives proposees

La section **Prediction Interactive** permet a l'utilisateur de saisir les parametres d'un ordre de travail (site, famille, cout, urgence, etc.) et d'obtenir trois predictions simultanees : risque de panne, besoin en pieces de rechange, et score d'anomalie. Les modeles sont charges via `@st.cache_resource` pour un chargement unique. Les features manquantes (les modeles attendent 62–67 features) sont completees par les medianes historiques.

6.3 Deploiement

L'application est deployee sur **Streamlit Community Cloud**, connectee a un depot GitHub prive. Les donnees traitees et les modeles entraines (.pkl) sont inclus dans le depot.

Contrainte de portabilite : Les fichiers pickle sont sensibles a l'environnement. Deux problemes ont ete rencontres :

1. **NumPy** : Les fichiers .pkl generes avec une version de NumPy provoquent des erreurs `ModuleNotFoundError: numpy._core.numeric` si charges avec une version differente. *Solution* : fallback systematique vers CSV pour les donnees.
2. **scikit-learn / Cython** : Le `GradientBoostingClassifier` serialise des objets Cython compiles (`CyHalfBinomialLoss`) qui sont specifiques a l'OS. Un modele entraine sous Windows ne peut pas etre deserialise sous Linux. *Solution* : re-entrainement a la volee au demarrage de l'application (~2 secondes pour 2035 echantillons), mis en cache par `@st.cache_resource`.

Les modeles Random Forest (panne, anomalies) et leurs scalers sont charges normalement depuis les fichiers .pkl, car ils ne contiennent pas d'objets Cython. La version de scikit-learn est epinglee a `==1.5.1` dans `requirements.txt`.

7 Recommandations

Chaque recommandation est directement derivee des analyses realisees sur les donnees.

Constat (donnees)	Recommandation	Impact es- time	Source
268 anomalies = 67.8% du cout correctif total	Revue systematique des ordres >10 000 MAD	~6 M MAD d'economies (reduction 50%)	Detection anomalies (IF+SVM)
81 equipements a 3+ pannes = 22.3% du cout	Analyse causes racines, remplacement si cout cumule > cout remplacement	~1.2 M MAD	Analyse recurrence
MTBF securite/closure : 7-8 jours	Inspection renforcee pour familles a MTBF <15j, remplacement clotures	Reduction pannes repetitives	Calcul MTBF
Ecart prestataires : x160 (30h vs 4873h)	Renegociation contrats >500h, redistribution volume	Reduction delais cloture	Performance prestataires
49% ruptures stock, 7.7M MAD affectes	Stock securite sur 144 articles Pareto, revoir seuils reappro	Reduction ruptures 49%→25%	Pareto + ruptures
Conformite preventive : 26.5% (TM2 : 6.3%, TMSA : 4.7%)	Suivi temps reel, penalites retard >7j	Objectif 60%	Conformite par site

TABLE 8 – Recommandations operationnelles avec impact chiffre

8 Limites et perspectives

8.1 Limites

- **Volume temporel** : 89 jours de données limitent la robustesse des prévisions et empêchent la modélisation de saisonnalités.
- **Absence de données capteur** : Les modèles se basent uniquement sur les ordres de travail historiques, sans télémétrie équipement (vibrations, température, etc.).
- **Target leakage potentiel** : La feature `equipment_failure_count` domine la prédiction de panne (37%). Cette information est disponible à l'entraînement car calculée sur l'historique complet, mais en production, seul l'historique passe serait disponible. Le risque est atténué par la validation croisée stable ($AUC_{CV} = 0.971$).
- **Déséquilibre des classes** : 18.9% de positifs pour les pannes, 26.2% pour les pièces. Le `class_weight='balanced'` compense partiellement.

8.2 Perspectives

- Intégration de données capteurs IoT pour une maintenance prédictive basée sur l'état réel des équipements
- Extension de la période de données pour capturer les tendances saisonnières
- Ajout de modèles de régression (coût, durée) en complément des classifieurs
- Mise en place d'un système d'alertes automatiques basé sur les prédictions du modèle
- Connexion directe avec le système GMAO pour des prédictions en temps réel

9 Conclusion

Ce projet a permis de construire un système complet d'analyse et de prédiction pour la maintenance industrielle, partant de données brutes jusqu'à une application web déployée. Les modèles développés atteignent des performances élevées ($AUC > 0.97$ pour la prédiction de panne, $AUC > 0.99$ pour les pièces de rechange) et l'analyse exploratoire a mis en évidence plusieurs axes d'amélioration opérationnelle concrets, notamment la conformité préventive (26.5%) et la gestion des stocks (49% de ruptures).

L'architecture modulaire adoptée (`src/` avec API uniforme) permet la réutilisation des composants et le re-entraînement indépendant des modèles au fur et à mesure de l'accumulation de nouvelles données.

Stack technique

Composant	Technologie
Langage	Python 3.12
Donnees	pandas, NumPy, openpyxl
Machine Learning	scikit-learn 1.5.1
Visualisation	Plotly, Streamlit
Deploiement	Streamlit Community Cloud
Versioning	Git / GitHub (repo prive)